

Name : Fahad Ali

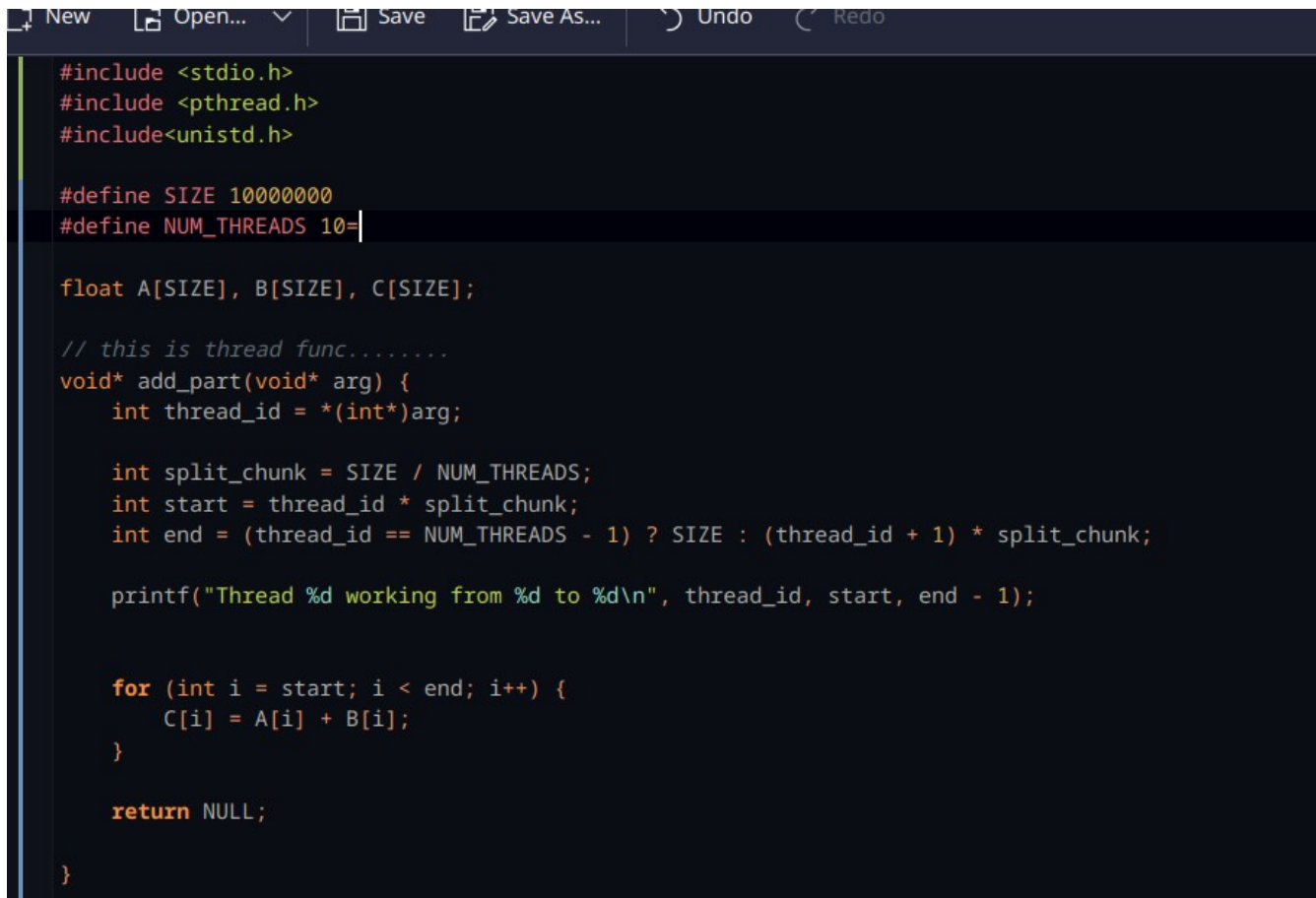
Roll no : 23P-0569

Section: CS-4k

In Lab Task-8

Task:1

Code Snapshots:



```
New Open... Save Save As... Undo Redo

#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

#define SIZE 10000000
#define NUM_THREADS 10

float A[SIZE], B[SIZE], C[SIZE];

// this is thread func.....
void* add_part(void* arg) {
    int thread_id = *(int*)arg;

    int split_chunk = SIZE / NUM_THREADS;
    int start = thread_id * split_chunk;
    int end = (thread_id == NUM_THREADS - 1) ? SIZE : (thread_id + 1) * split_chunk;

    printf("Thread %d working from %d to %d\n", thread_id, start, end - 1);

    for (int i = start; i < end; i++) {
        C[i] = A[i] + B[i];
    }

    return NULL;
}
```

```

int main() {

    pthread_t threads[NUM_THREADS];    //threads....
    int thread_ids[NUM_THREADS];

    printf("Initializing arrays...\n");
    sleep(1);

    for (int i = 0; i < SIZE; i++) {
        A[i] = i * 1.0;
        B[i] = i * 2.0;
    }

    printf("Now Starting paralel computation...\n");
    sleep(1);

    // Here Creating threads.....
    for (int i = 0; i < NUM_THREADS; i++) {
        thread_ids[i] = i;
        pthread_create(&threads[i], NULL, add_part, &thread_ids[i]);
        sleep(1);
    }

    for (int i = 0; i < NUM_THREADS; i++) {    // Joining threads.....
        pthread_join(threads[i], NULL);
    }

    printf("Parallel computation completed!\n");

    printf("\nOutput:\n");
    for (int i = 0; i < 5; i++) {
        printf("C[%d] = %.2f\n", i, C[i]);
    }

    return 0;
}

```

Output:

```
fkhokar@fedora:~/Desktop/In-Lab-8$ gcc Task1.c
fkhokar@fedora:~/Desktop/In-Lab-8$ ./a.out
Initializing arrays...
Now Starting paralel computation...
Thread 0 working from 0 to 999999
Thread 1 working from 1000000 to 1999999
Thread 2 working from 2000000 to 2999999
Thread 3 working from 3000000 to 3999999
Thread 4 working from 4000000 to 4999999
Thread 5 working from 5000000 to 5999999
Thread 6 working from 6000000 to 6999999
Thread 7 working from 7000000 to 7999999
Thread 8 working from 8000000 to 8999999
Thread 9 working from 9000000 to 9999999
Parallel computation completed!

Output:
C[0] = 0.00
C[1] = 3.00
C[2] = 6.00
C[3] = 9.00
C[4] = 12.00
fkhokar@fedora:~/Desktop/In-Lab-8$
```

Task:2

Code Snapshots:

```
> Task1.c x Task2.c x
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>

int Avg;
int Min;
int Max;

// these is Thread functions.....s
void* find_average(void* arg) {
    int* arr = (int*)arg;
    int sum = 0;
    int n = arr[0];    // first value = size

    for (int i = 1; i <= n; i++) {
        sum += arr[i];
    }

    Avg = sum / n;
    printf("Average thread calculated sum = %d and average = %d\n", sum, Avg);
    pthread_exit(0);
}

void* find_minimum(void* arg) {
    int* arr = (int*)arg;
    int n = arr[0];

    Min = arr[1];

    for (int i = 2; i <= n; i++) {
        if (arr[i] < Min) {
            Min = arr[i];
        }
    }

    printf("Minimum thread found min = %d\n", Min);
    pthread_exit(0);
}
```

```

void* find_maximum(void* arg) {
    int* arr = (int*)arg;
    int n = arr[0];

    Max = arr[1];

    for (int i = 2; i <= n; i++) {
        if (arr[i] > Max) {
            Max = arr[i];
        }
    }
    printf("Maximum thread found max = %d\n", Max);

    pthread_exit(0);
}

int main(int argc, char* argv[]) {

    // Check input
    if (argc < 2) {
        printf("Please enter numbers on command line.\n");
        return 1;
    }

    int n = argc - 1;

    // Create array (extra space to store size at index 0)
    int* numbers = (int*)malloc((n + 1) * sizeof(int));

    numbers[0] = n;

    // Convert input strings to integers

```

```

pthread_t t1, t2, t3;

printf("Calculating using threads...\n");
sleep(1);

// Create threads
printf("Creating threads...\n");
sleep(1);
pthread_create(&t1, NULL, find_average, numbers);
pthread_create(&t2, NULL, find_minimum, numbers);
pthread_create(&t3, NULL, find_maximum, numbers);

pthread_join(t1, NULL);
pthread_join(t2, NULL);
pthread_join(t3, NULL);

printf("\nResults:\n");
printf("The average value is %d.\n", Avg);
printf("The minimum value is %d.\n", Min);
printf("The maximum value is %d.\n", Max);

free(numbers);

return 0;
}

```

Output:

```

fkhokar@fedora:~/Desktop/In-Lab-8$ gcc Task2.c -o Task2 -lpthread
fkhokar@fedora:~/Desktop/In-Lab-8$ ./Task2 90 81 61 12 30 40 43
Calculating using threads...
Creating threads...
Average thread calculated sum = 357 and average = 51
Minimum thread found min = 12
Maximum thread found max = 90

Results:
The average value is 51.
The minimum value is 12.
The maximum value is 90.
fkhokar@fedora:~/Desktop/In-Lab-8$ 

```

